
LemmaPortfolio: Anonymous Technical Supplement

Anonymous Author(s)

Affiliation

Address

email

1 This supplement specifies the benchmark, construction, response transcription, and artifact trail
2 behind the four-page paper. It is self-contained for score reproduction: every score-bearing claim
3 can be traced from the supplied response PDFs and episode-key transcriptions to a public episode, a
4 committed label row, and deterministic scoring code. The accompanying anonymous archive contains
5 those files and the verification commands. It also records what the evidence does *not* establish.

6 1 Task definition

7 For each episode, let C be the displayed candidate set, T the displayed target set, and B the selection
8 budget. The released benchmark fixes $|C| = 16$, $|T| = 8$, and $B = 3$. Candidate and target identifiers
9 are local to an episode; a name such as C04 has no meaning in another episode. Candidate and target
10 declaration names are replaced by these identifiers, but the canonical candidate types and target
11 statements remain visible.

12 For each target t , the label records

$$D(t) = C \cap \text{Consts}(\text{value}(t)),$$

13 where `value` is the elaborated Lean declaration value (including opaque theorem values in the
14 audit) and `Consts` returns the constants occurring directly in its expression tree. The intersection is
15 important: a target may use other declarations that are not among the 16 displayed candidates. No
16 transitive closure is taken.

17 For an exact-budget portfolio $S \subseteq C$, define

$$F(S) = \sum_{t \in T} \mathbf{1}[D(t) \cap S \neq \emptyset].$$

18 The scorer enumerates every $S \subseteq C$ with $|S| = B$ in lexicographic order, computes $F^* =$
19 $\max_S F(S)$, and reconstructs the complete set $\mathcal{A} = \{S : F(S) = F^*\}$. A response is correct
20 precisely when it is a valid size- B subset in $\mathcal{A}$. The stored optimum and stored argmax list are treated
21 as assertions to audit, not as authority: scoring stops if either differs from the fresh enumeration.

22 The three secondary item scores are raw target coverage $F(S)$ (reported in aggregate also as
23 $F(S)/|T|$),

$$\text{normalized coverage} = F(S)/F^*, \quad \text{edge recall} = \frac{\sum_t |D(t) \cap S|}{\sum_t |D(t)|}.$$

24 A missing, malformed, wrong-budget, duplicate-ID, or out-of-pool response gets zero on the primary
25 exact-optimum score and all three secondary scores. Validity and operational status are reported
26 separately.

27 1.1 Exact uniform-random reference

28 There are $\binom{K}{B}$ equally likely portfolios for $K = |C|$. Hence, for one episode,

$$\Pr(S \text{ is optimal}) = \frac{|\mathcal{A}|}{\binom{K}{B}}.$$

For a target with $d_t = |D(t)|$, a random portfolio misses all its displayed dependencies with probability $\binom{K-d_t}{B} / \binom{K}{B}$ (taken as zero when $K - d_t < B$). Linearity of expectation therefore gives

$$\mathbb{E}[F(S)] = \sum_{t \in T} \left(1 - \frac{\binom{K-d_t}{B}}{\binom{K}{B}} \right).$$

We average the item-level optimal probabilities and $\mathbb{E}[F(S)]/F^*$. Since each displayed edge is incident to one candidate, its selection probability and the expected edge recall are both $B/K = 3/16$. No Monte Carlo estimate is needed. A single seeded SHA-256 draw is retained only as a parser and provenance check.

2 Source snapshot and episode construction

2.1 Pinned source

The construction begins from MATHLIB v4.33.0 [The mathlib Community, 2026]. Its pinned Git commit is

db584cd6d46c92f209a44c0f1c829460d327499d.

The extraction consists of 256 Parquet shards from the Mathlib Initiative dependency and type datasets [Mathlib Initiative, 2026a,b]. Before mining, the builder recomputes the name, byte size, and SHA-256 digest of every shard and compares the complete inventory with the earlier source receipt. It refuses a missing, extra, reordered, resized, or changed shard. The generation receipt records the complete inventory rather than relying on a directory-level label.

Score reproduction uses the released episode and label JSONL files and does not require these Parquet shards. The pinned-Lean audit can recompute every displayed edge from the named MATHLIB commit. Repeating the original mining search, however, additionally requires the separately archived 256-shard snapshot; its per-file inventory detects alteration but cannot recover missing bytes.

Eligible declarations have a nonempty, parseable public type, a source module, and a proof-body dependency record. We remove source modules used by any earlier LemmaPortfolio prototype and reject any statement whose whitespace-normalized or leading-binder- α -normalized signature appeared in those prototypes. The same two signatures are used to forbid duplicates within and between the new development and blind splits.

2.2 Cohorts, targets, and masks

Each episode is centered on one source module. Candidate cohorts are drawn from declarations that recur as dependencies elsewhere in the corpus. Targets come from the same module and have at least one edge to the cohort. Stable SHA-256 ranks, fixed in the builder, determine cohort trials, target trials, candidate display order, target display order, and final split order. Human or model responses are not inputs to any of these ranks.

The public row contains only the benchmark ID, local candidate and target IDs, canonical statement strings, budget, task text, and source-project label. The released label row maps local IDs back to source declarations and records every $D(t)$, candidate use counts, F^* , and every optimal portfolio. Before accepting a row, the builder rejects any source theorem name visible in any displayed statement. It also rejects exact or α -equivalent candidate-candidate, candidate-target, and target-target collisions.

2.3 Structural stress filters

Table 1 lists the fixed filters applied before final split selection. They were chosen on development experience and then applied identically to development and blind candidates. A candidate is active if it belongs to at least one $D(t)$; “near-optimal” means coverage $F^* - 1$.

The miner further requires 20 fixed methods to miss the optimum. Eighteen use only public strings: candidate position or length ranks, token-count and connective ranks, aggregate candidate-target text similarity, and three token-Jaccard inferred-incidence widths. Two use the true incidence graph:

Table 1: Episode-level construction requirements.

Requirement	Threshold
Displayed candidates / targets / budget	16/8/3
Active candidates	at least 7
Active candidates outside the union of all optima	at least 4
Targets with two or more displayed direct dependencies	at least 2
Coverage of any singleton candidate	at most 4 of 8
Use count of every member of an optimal portfolio	at least 2
Loss after removing any member of an optimal portfolio	at least 1 target
Distinct portfolios at coverage $F^* - 1$	at least 2
Optimal coverage	at least 6 of 8
Number of accepted optimal portfolios	at most 32
Mean pairwise target token-Jaccard similarity	at most 0.6

frequency rank and greedy maximum coverage. This conditioning is disclosed in the public generation receipt. These methods describe how the challenge set was selected and must not be presented as held-out baselines.

The post-freeze character TF-IDF baseline is different in both timing and implementation. It is specified before the final commitment but generated afterward, reads public statements only, and never enters item admission. It uses character n -grams for $n \in \{3, 4, 5\}$, sublinear term frequency $1 + \ln c$, smoothed inverse document frequency $\ln((1 + N)/(1 + \text{df})) + 1$, fitted within each episode over its 24 candidate and target statements, L2 normalization, and cosine similarity. It enumerates all size-three portfolios to maximize $\sum_t \max_{c \in S} \text{sim}(t, c)$, breaking ties lexicographically. Because both methods use surface text, we still describe it as related to the lexical diagnostic family.

Table 2: Final split summary after the pinned-Lean replacement.

	Development	Blind
Episodes / distinct modules	30 / 30	60 / 60
Top-level domains	13	15
Displayed targets	240	480
Displayed direct edges	386	814
Multi-edge targets	99	226
Mean active candidates	8.37	8.60
$F^* = 6 / F^* = 7$	25 / 5	45 / 15
Accepted optimal portfolios	30	78
Construction-diagnostic hits	0	0

The miner began with 350,599 declarations. After eligibility and prior-split exclusions, 186,680 declarations remained across 7,089 module centers. Stable cohort and target trials produced 71,091 structurally complete incidence drafts. Of these, 165 made all 20 construction diagnostics miss, 121 also passed name masking, and 103 also passed within-episode statement identifiability. One initially selected blind module failed the independent Lean edge audit and was excluded by its module hash; the final row was chosen deterministically from the already ranked admissible superset.

The final blind domain counts are: Algebra 10, Data 7, Ring Theory 7, Analysis 5, Topology 5, Algebraic Geometry 4, Category Theory 4, Combinatorics 3, Number Theory 3, Order 3, Dynamics 2, Geometry 2, Group Theory 2, Linear Algebra 2, and Measure Theory 1. The largest domain therefore contributes 10/60 episodes.

3 Development policy and dataset commitment

Thirty development episodes are partitioned in manifest order into two predeclared, nonoverlapping batches of 15. Batch A supplies the sole model-facing difficulty check: a controlled Codex CLI Sol capture obtained 6/15 exact optima (40%). This single check suggested that the revised format was nontrivial. It is development evidence only, not a consumer-chat test result, an upper bound, or a guarantee that blind Sol performance will be below 60%. Batch B is not used for additional model

99 calibration, and neither batch is used to choose the ordered-support width. No development item
100 is substituted for a blind item. All development labels are public and excluded from blind prompt
101 directories.

102 The 60 test episodes are selected by the same structural rules from disjoint modules, with a fixed
103 top-level-domain cap. The source theorems remain public; “hidden” refers to the newly composed
104 episode and its incidence row. On 1 September 2026, before the response captures, the project
105 recorded the public test SHA–256 and label-file commitment shown below.

106 b54293b08bc7ddb7110a50908049922f7cfd86f14cb49a4de4e28353aecc6d8
107 d17c1fd5c6d87635daf8035fe4a0918d1eaa29708eae37cb797cae08c19ef887

108 The same receipt binds aggregate results from the pinned-Lean edge audit. The model-facing prompts
109 contain the public rows, never the incidence labels.

110 An earlier protocol anticipated a fully frozen manual panel with per-call timestamps, attestations,
111 and closure receipts. The material actually supplied for this study is two PDFs of visible responses,
112 not those richer capture forms. We do not manufacture the missing evidence or inherit claims
113 from the unused protocol. The evaluation therefore supports deterministic score reproduction and a
114 prior dataset commitment, but not independent verification of fresh-chat use, interface settings, call
115 chronology, or the exact serving checkpoints.

116 4 Pinned-Lean replay audit

117 The source Parquet graph is convenient for mining but is not accepted as its own verification. A
118 second program builds one Lean input stream in a clean MATHLIB checkout and imports Mathlib.
119 For every displayed source declaration it checks that the constant exists and has the expected
120 declaration kind. For every target it reads `ConstantInfo.value?` with opaque values enabled,
121 obtains `Expr.getUsedConstantsAsSet`, intersects that set with the 16 displayed candidate names,
122 and compares the result exactly with the stored row. It also repeats the cross-statement `Expr.eqv`
123 checks.

124 The Python wrapper binds the public and label commitments, checks the clean MATHLIB commit,
125 verifies Lean 4.33.0 and its compiler commit, and emits only aggregate counts. This keeps private
126 declaration names out of a public audit log. Negative tests mutate a label edge, each commitment, the
127 source inventory, and the environment; every mutation must fail closed.

Table 3: Aggregate pinned-Lean audit of final bytes.

Check	Development	Blind
Episodes	30/30	60/60
Targets re-extracted exactly	240/240	480/480
Stored / re-extracted displayed edges	386/386	814/814
Changed targets	0	0
Added / removed edges	0/0	0/0
Empty target incidence	0	0
Expression-equivalent statement collisions	0	0

128 The audit verified clean Mathlib commit db584cd6d46c92f209a44c0f1c829460d327499d, Lean
129 4.33.0 compiler commit d8b18978322de05a8f3dba51ef03cf5461676c17, all expected declara-
130 tion kinds, and exact public/private commitments. A separate static audit re-hashed all 256 Parquet
131 shards and reproduced every final development and blind output byte-for-byte.

132 5 Response intake and scoring protocol

133 5.1 Direct rows

134 The score sources are a 15-page PDF entitled *Models and their responses* and a separate four-page
135 PDF headed *Gpt sol 5.6.xhigh*. Together their section headings supply the six visible consumer labels
136 used in the paper. The combined document contains twelve intended five-episode direct responses

per row, except for the two anomalies described below; the xhigh PDF contains all twelve. Neither contains account-tier, system-prompt, token-usage, latency, screenshot, or provider-log evidence. Accordingly, we make no claim about an underlying API checkpoint and do not infer operational facts that are absent from the document.

Transcription removes layout whitespace that the PDF extractor inserts inside identifiers, decodes complete JSON envelopes, and indexes selections by explicit MLP4B_xxxx keys. The expected key ranges are D01=0000–0004 through D12=0055–0059. Visual order is irrelevant. Every selection must contain three distinct in-pool IDs. A missing or malformed whole response block gives all five episodes zero while retaining them in the denominator. This strict rule avoids silently improving an output by editorial repair. The canonical machine-readable transcriptions and both source-document hashes are released beside the scorer.

The Pro section of the combined PDF contains D01–D11, a second D11 response, and no D12. We retain the first D11 occurrence; the only changed item in the second version has identical exact and secondary scores, so this convention does not affect any reported number. Qwen 3.8 contains the D12 keys but omits the envelope’s outer opening brace. We count that entire block invalid; adding the brace would increase its exact score from 12/60 to 15/60, but that repaired value is not reported. All other combined-PDF blocks decode and align uniquely by episode key. The separate xhigh direct blocks decode once each in D01–D12 order and all 60 selections are valid.

5.2 Ordered-support diagnostic

Each GPT SOL response section provides four additional ranked-support blocks, covering every target in D04, D05, D06, and D12. The analysis truncates each list to its first $q = 2$ entries and enumerates all 560 portfolios against the predicted incidence, selecting the lexicographically first maximizer. This postprocessor uses no gold label. After label release, full and truncated support maps are compared with the direct-incidence graph using micro precision, recall, and F1. The same 20 episode IDs permit a descriptive comparison with direct prompting. Because the PDFs do not establish fresh-chat use, counterbalanced chronology, or an absence of intervening feedback, we do not call the difference a causal prompting effect and report no significance test.

6 Complete results

6.1 Complete score table

Row	Exact	Valid	Mean $F/8$	Mean F/F^*	Edge recall
SHA–256 deterministic draw	0/60	60/60	2.30/8	37.1%	18.9%
Character TF–IDF	0/60	60/60	3.77/8	60.4%	29.5%
GPT SOL 5.6 Pro	25/60	55/60	4.98/8	79.6%	38.9%
GPT SOL 5.6 (xhigh)	18/60	60/60	5.08/8	81.4%	39.8%
DeepSeek Instant + DeepThink	1/60	60/60	4.20/8	67.1%	34.9%
DeepSeek Expert + DeepThink	7/60	60/60	4.60/8	73.4%	36.9%
Qwen 3.8 Max–Thinking	12/60	55/60	4.32/8	68.9%	34.0%
Qwen 3.7 Plus–Thinking	6/60	60/60	4.05/8	64.6%	31.8%
GPT SOL 5.6 Pro support $q = 2$	1/20	20/20	4.60/8	74.8%	34.5%
GPT SOL 5.6 (xhigh) support $q = 2$	0/20	20/20	3.90/8	63.8%	31.3%

Table 4: Complete fixed-denominator scores. Invalid responses contribute zero to every metric. The analytic uniform-random expectation is 0.14 exact episodes, 2.35/8 mean coverage, 37.6% mean F/F^* , and 18.75% edge recall.

For the six consumer rows, item-wise Wilson 95% intervals for exact accuracy are, in table order, 30.1–54.3%, 19.9–42.5%, 0.3–8.9%, 5.8–22.2%, 11.8–31.8%, and 4.7–20.1%. Resampling the twelve five-item response blocks gives descriptive 95% percentile intervals of 30.0–53.3%, 15.0–45.0%, 0.0–5.0%, 5.0–18.3%, 8.3–33.3%, and 1.7–20.0%. These intervals summarize sensitivity within this fixed challenge set; they are not estimates for model families.

6.2 Block-level exact scores and response alignment

The source document does not always present blocks in numerical order. The alignment is recovered from the explicit episode keys. The Pro section of the combined PDF supplies

Row	D01	D02	D03	D04	D05	D06	D07	D08	D09	D10	D11	D12
GPT SOL 5.6 Pro	2	2	2	2	1	1	4	2	3	3	3	0
GPT SOL 5.6 (xhigh)	0	1	3	1	0	0	4	3	2	2	2	0
DeepSeek Instant	0	0	0	0	0	0	0	0	1	0	0	0
DeepSeek Expert	1	0	0	0	1	0	1	1	0	1	2	0
Qwen 3.8 Max	1	1	1	0	0	0	1	3	3	0	2	0
Qwen 3.7 Plus	0	0	0	0	0	0	1	2	1	0	0	2

Table 5: Exact optima per five-episode direct block. Each row sums to the main-table numerator.

D01–D11, then a second D11 variant, and no D12. Both D11 variants score 3/5 and have the same secondary metrics; the first is retained as the canonical transcription. The separate xhigh PDF supplies D01–D12 in numerical order. DeepSeek Instant and Expert supply D01–D12. Qwen 3.8 supplies D01,D03,D02,D04,D06,D05,D08,D07,D09,D10,D12,D11; its D12 text is malformed because the outer opening brace is absent. Qwen 3.7 supplies D02,D01,D03,D04,D08,D07,D10,D09,D05,D06,D12,D11. No positional guess is used.

6.3 Structured-support diagnostic

Each GPT response source has four support blocks covering D04, D05, D06, and D12. For Pro, the fixed $q = 2$ postprocessor produces exact counts 1, 0, 0, 0, compared with direct counts 2, 1, 1, 0. Full support has micro precision/recall/F1 of 72.5/75.6/74.0%; at $q = 2$ the figures are 76.8/68.7/72.6%. For xhigh, the structured counts are 0, 0, 0, 0 versus direct 1, 0, 0, 0. The Pro direct comparator has 15/20 valid outputs, whereas its structured arm and both xhigh arms have 20/20. The xhigh full-support figures are 69.3/76.4/72.7%, and its $q = 2$ figures are 71.8/66.5/69.1%. Because neither PDF certifies paired-call chronology, these are descriptive pipeline comparisons rather than causal prompt ablations.

6.4 Provenance boundary

The public test file and label digest were recorded on 1 September 2026. The score sources are the combined response PDF and the separately supplied xhigh PDF. Their SHA–256 digests, after those of the public file and labels, are:

```
b54293b08bc7ddb7110a50908049922f7cfd86f14cb49a4de4e28353aecc6d8
d17c1fd5c6d87635daf8035fe4a0918d1eaa29708eae37cb797cae08c19ef887
2f20fbc4b3eb3b87c4b693058fc5744a947de04e19a2eca70f0ea5c0d54dde3f
02abb92d1e3fd6d59dca5faa191a7fec214dffae7c4f023ce5cbb5d02b517afa
```

The archive preserves both PDFs, the episode-key transcriptions, and the scoring outputs. It does not contain provider-side request logs, per-chat timestamps, or independently verified deployment screenshots. We therefore describe the rows as externally captured consumer responses with a pre-existing dataset commitment, not as checkpoint-reproducible or cryptographically escrowed runs.

7 Design history: excluded earlier captures

Earlier LemmaPortfolio prototypes are not pooled with this benchmark. They used smaller candidate pools, a two-item budget, one privileged exact pair, and post-release inspection. Their web-interface captures and local quantized-model captures differ in routing, prompting, context allocation, runtime, and item set. In particular, the preserved evidence for an earlier 21/30 development capture does not establish the GPT-4 identity originally associated with it. We retain the number only as design history and make no GPT-4 result claim.

Likewise, an old local row labelled “DeepSeek” sent 15 episodes through one shared reasoning budget and repeatedly fell back after parser failures. It is not comparable to a fresh consumer-web deployment and is not evidence that the DeepSeek family scores two points. We also explored

212 fresh-item local Q4 captures while redesigning the protocol, but they use a different inference surface
213 and do not enter the final panel. No earlier score or response is reused.

214 **8 Interpretation and limitations**

215 The benchmark has four main boundaries.

216 First, $D(t)$ is historical. A declaration can be logically useful without appearing in the stored proof,
217 and a stored dependency can be replaceable. The task therefore measures dependency compression,
218 not causal utility. UseFor, for example, studies reusability and contribution to proof likelihood in a
219 self-play setting [Kasriel et al., 2025]; that is a complementary and stronger utility question.

220 Second, this is a closed pool with uniform costs. Real systems search much larger libraries and
221 may assign different context or retrieval costs to declarations. The coverage objective is monotone
222 submodular, connecting the task to classical maximum-coverage optimization [Nemhauser et al.,
223 1978], but $K = 16$ and $B = 3$ let us enumerate exactly rather than rely on approximation guarantees.

224 Third, the stress split is conditioned to defeat fixed diagnostics. Its score is useful for comparing
225 declared methods on this challenge distribution, not for estimating the frequency of hard modules
226 in all of MATHLIB. Raw counts are primary. Item-wise Wilson intervals show binomial-scale
227 variation under an exchangeable-item working model, while whole-block bootstrap intervals show
228 sensitivity to resampling the fixed prompt groups. Neither estimates population performance or
229 model-generation variance. The 20-item intervention has only four supplied five-item blocks and
230 supports no significance claim.

231 Fourth, consumer-chat deployments hide routing, system prompts, and decoding, and the supplied
232 PDFs contain no per-call metadata. The rows should not be read as a provider or architecture ranking.
233 There is no human-expert baseline, and no controlled repeated sampling is available for estimating
234 generation variance. Codex agents assisted benchmark design, evaluation code, response transcription,
235 and manuscript development; the authors checked the scores and remain responsible for every claim.

236 **9 Artifact map and reproduction**

237 The anonymous archive is organized so that score reproduction is distinct from the more expensive
238 reconstruction of the benchmark:

239 `data/` development and test public rows and released labels;

240 `responses/` both source PDFs, canonical direct and support transcriptions, and the align-
241 `ment/anomaly` ledger;

242 `results/` fixed baseline selections and machine-readable aggregate, item, and block scores;

243 `provenance/` the dataset commitment and aggregate construction and pinned-Lean receipts;

244 `development_evidence/` the 15-item difficulty capture and its decision receipt;

245 `construction/` the benchmark builder, tests, and pinned-Lean audit wrapper;

246 `tools/` a standard-library scorer and manifest builder;

247 `verify_release.py` the one-command release verifier; and

248 `paper/` anonymous sources and the checked anonymous PDFs.

249 The one-command verifier uses only the Python standard library. It checks the published hashes,
250 reconstructs all optima from the released incidence rows, and recomputes every response-row,
251 baseline, and development-check score without calling a model, Lean, or a paid service. The archive
252 also supplies the pinned-Lean audit wrapper and tests. Repeating that extraction additionally requires
253 Lean 4.33.0 and the named MATHLIB snapshot; the original mining search requires the separately
254 obtainable 256 Parquet shards. Exact commands and a short path for a nontechnical collaborator are
255 in the archive README.

References

- Timothe Kasriel, Thomas Lu, Devon Ding, Jingxuan He, and Dawn Song. Usefulness-driven learning of formal mathematics. In *NeurIPS Workshop on Mathematical Reasoning and AI*, 2025. URL <https://openreview.net/pdf/023eb56eadd5685c15a6489e0fddd540279b8c45.pdf>.
- Mathlib Initiative. Mathlib constant dependencies. Hugging Face dataset, 2026a. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-const-dep>. Version 4.33.0; exact 256-shard inventory recorded in the artifact.
- Mathlib Initiative. Mathlib types. Hugging Face dataset, 2026b. URL <https://huggingface.co/datasets/mathlib-initiative/mathlib-types>. Version 4.33.0; exact 256-shard inventory recorded in the artifact.
- George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971.
- The mathlib Community. mathlib4 source snapshot for version 4.33.0. Git repository commit, 2026. URL <https://github.com/leanprover-community/mathlib4>. Commit db584cd6d46c92f209a44c0f1c829460d327499d.